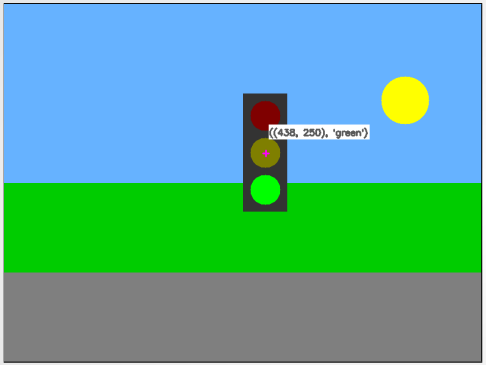
Computer Vision
Fall 2025
Problem Set #2

Divyesh Baskaran
dbaskaran6@gatech.edu

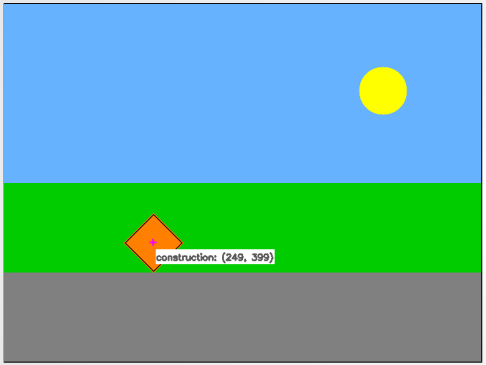
1a)Traffic Light Detection



ps2-1-a-1

Coordinates and State:
(438, 250), color: green

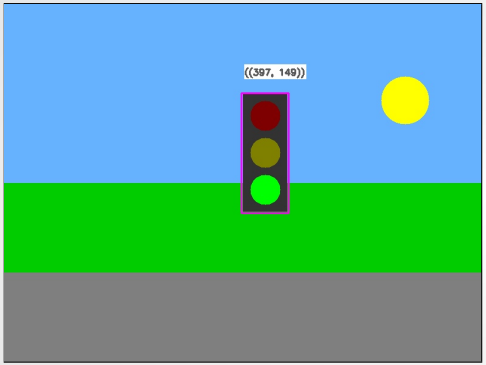
1b)Traffic Sign Detection - Construction



ps2-1-b-1

Coordinates:
(249, 399), color: orange

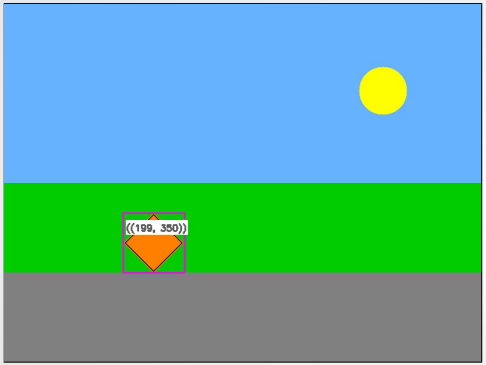
2a)Template Matching - TL



ps2-2-a-1

Coordinates:
(397, 149)

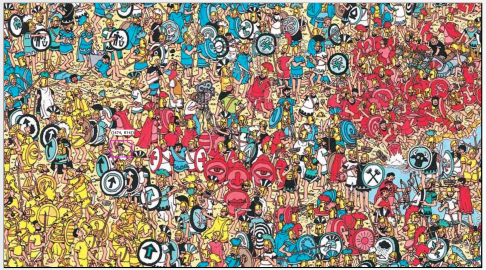
2b)Template Matching - Construction



ps2-2-b-1

Coordinates:
(199, 350)

2c)Template Matching - Finding Waldo



ps2-2-c-1

Coordinates:
(474, 614)

2d)Discussion

What are the disadvantages of using Hough based methods in finding Waldo? Can template matching be generalised to all images?

Explain Why/Why not. Which method consistently performed the best, why?

Hough transforms just aren't built for finding Waldo. This method is looking for exact parameters, stuff like lines (ρ, θ) or circles (x, y, r), and Waldo doesn't really have a parametric shape. He's basically just a random pattern of clothes and accessories. Even if you tried to match circles for his glasses, the scene is so crowded that the accumulator would end up with a ton of false peaks. And on top of that, Hough voting ignores color completely and takes forever to run on a big cluttered image.
Template matching isn't much better honestly. It assumes the target is the exact same scale and rotation as the template, so it falls apart the second there's a lighting change or something's occluded. The Waldo image also has way too many repeating patterns that end up scoring higher than the actual match. Basically this whole thing is a lot harder than the synthetic images from Part 1, since those objects were just sitting there isolated with nothing around them.

Out of the three metrics, `tm_ssd` and `tm_nssd` worked the best, they nailed the traffic light, the construction sign, and Waldo every time.

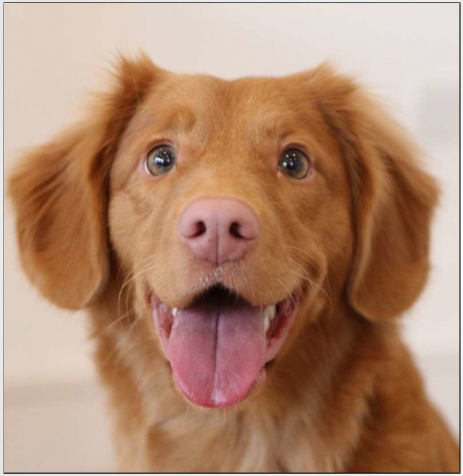
Makes sense since they're just minimizing raw pixel differences, and Waldo's colors are pretty distinct from everyone around him. `tm_nccor`

on the other hand was all over the place. It drifted off the construction sign and totally whiffed on Waldo, locking onto some random red and

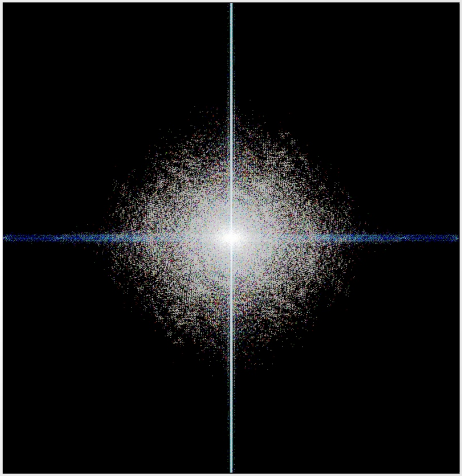
black soldiers instead. I think the normalization is the problem, it makes the metric care more about local contrast than actual color, which

kills it in a scene this repetitive.

4a1) Compression - Threshold 0.1

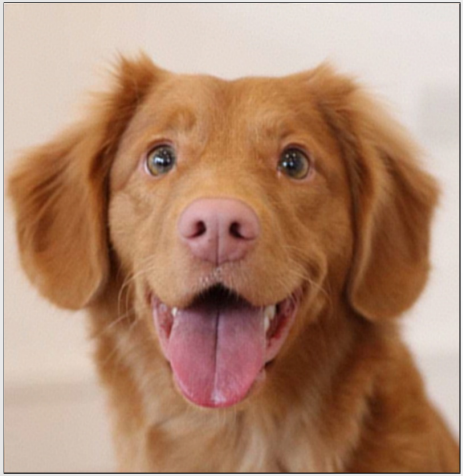


ps2-4-a-1 resulting image

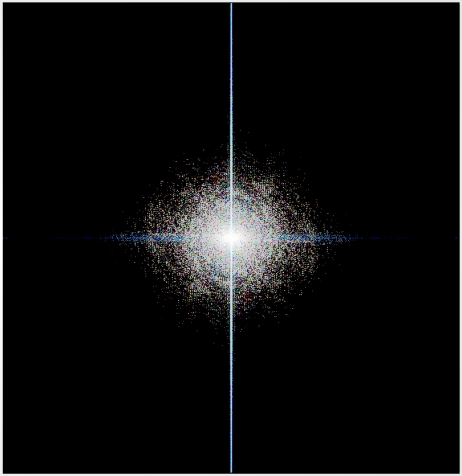


ps2-4-a-1 frequency domain

4a2) Compression - Threshold 0.05



ps2-4-a-2 resulting image

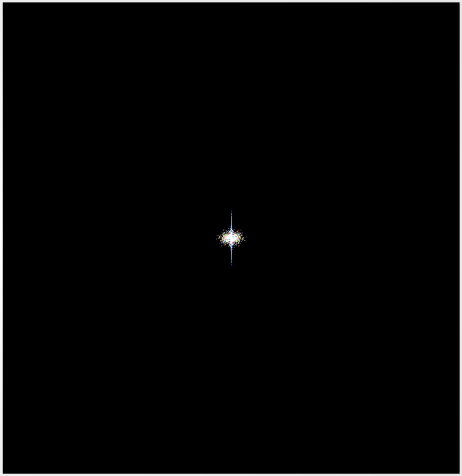


ps2-4-a-2 frequency domain

4a3) Compression - Threshold 0.001



ps2-4-a-3 resulting image

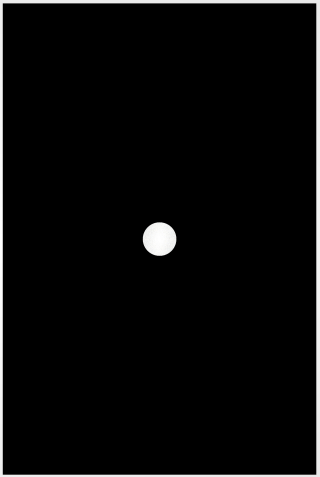


ps2-4-a-3 frequency domain

5a1) Filtering - Radius 100



ps2-5-a-1 resulting image

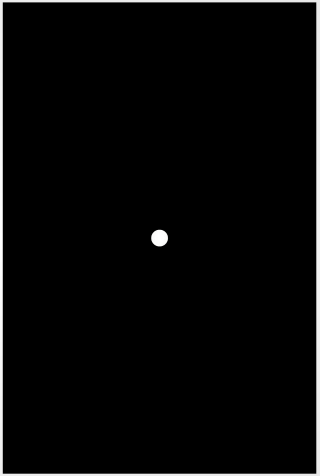


ps2-5-a-1 frequency domain

5a2) Filtering - Radius 50



ps2-5-a-2 resulting image

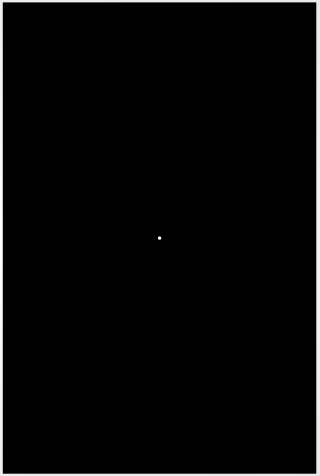


ps2-5-a-2 frequency domain

5a3) Filtering - Radius 10



ps2-5-a-3 resulting image



ps2-5-a-3 frequency domain

5b) Discussion

What are the differences between compression and filtering? How does this change the resulting image?

Compression and low-pass filtering both work in the frequency domain but pick their coefficients differently. Compression just ranks everything by magnitude and keeps whatever's strongest, doesn't matter what frequency it's at. Low-pass filtering instead keeps everything inside some fixed radius and zeros out the rest, even if that discarded stuff actually had a lot of energy in it.
Natural images tend to keep their important edge info in the high frequencies. Since compression keeps whatever has the highest magnitude, it ends up preserving those edges anyway. At threshold 0.001 we only kept 0.1% of the frequencies and the dog was still recognizable, just blocky. Low-pass filtering doesn't care about that though, it just chops high frequencies no matter what. Even at radius 10, which keeps more coefficients, the cat image ended up so blurred that all the fur texture was basically gone.

So overall: compression adapts to the actual content, so the image stays structurally sharp but gets blocky. Low-pass filtering just grabs a fixed band of frequencies, so the image gets more and more blurry as the radius shrinks.

5c) Discussion

Given an image corrupted with salt and pepper pepper noise, what filtering method can effectively reduce/remove this noise? Also explain your choice of filtering method.

Median filtering is honestly the way to go for salt-and-pepper noise. This kind of noise just flips random pixels to pure black or white, and the median filter fixes it by replacing each pixel with the median of its local window. As long as the noise is less than half the window, the filter basically just ignores those extreme values. Ends up removing the noise completely while keeping edges sharp.

A linear filter like Gaussian blur is not the move here though. It computes a weighted average, so those extreme noise values just get smeared across nearby pixels instead of removed. You end up with faint blotches everywhere plus a blurry image overall. Basically the median filter actually targets the outliers directly, while the linear filter just spreads the noise around instead of dealing with it.